

Sentry AI-Powered Laser Tracking Turret

1. Objective & Motivation:

This project proposes an active, autonomous "Eye-to-Hand" tethered robotic sentry turret that actively tracks human subjects and dynamically targets specific body parts (head, chest, or hand) using a laser pointer. The goal is to build a real-time system that proves the viability of combining modern deep learning with classical kinematics, utilizing a high-performance local compute node for complex vision processing and an embedded edge device for low-latency physical actuation.

2. Hardware Architecture & Embedded Components

To ensure a robust, low-latency tracking system, the physical architecture strictly divides computational heavy lifting from mechanical actuation. The hardware is structurally divided into processing, actuation, and power/data routing:

- **Central Compute Node (Vision & AI)**
 - **Local Processing Hub (Laptop/PC):** Serves as the primary vision sensor and AI processing engine. Utilizing an integrated HD Webcam and a dedicated GPU (e.g., RTX 4060), this hub captures the live feed, executes the deep learning pose-estimation pipeline, calculates the required inverse kinematics, and streams absolute physical angle commands over a high-speed serial tether.
- **Processing & Vision Hub**
 - **Edge Microcontroller (ESP32-CAM with OV2640 Sensor):** Repurposed strictly as a dedicated edge motor controller. It receives high-speed text commands via a physical serial tether and outputs stable Pulse Width Modulation (PWM) signals to drive the actuators, completely bypassing wireless network latency.
 - **Serial Interface (ESP32-CAM-MB Shield or FTDI Programmer):** Utilized to establish the serial communication protocol and flash the initial C++ firmware to the microcontroller prior to wireless deployment.
- **Actuation & Targeting Payload**
 - **2-DOF Actuators (2x SG90 Micro Servos):** Configured in a mechanical pan-tilt linkage to execute the X-axis (yaw) and Y-axis (pitch) movements. These

servos respond directly to the dynamic angle adjustments generated by the PID controller.

- **Targeting Indicator (5V Laser Diode):** Serves as the visual proof of targeting lock. It is mounted strictly parallel to the camera lens to eliminate parallax error and ensure spatial tracking accuracy.
- **Mechanical Chassis (1x SG90 Pan/Tilt Camera Bracket):** A structurally rigid, 3D-printed or standardized mounting bracket designed to house the actuators, camera, and laser payload as a singular moving unit.
- **Status Indicators (2x 5V LEDs):** Two colored LEDs (Red/Green) wired to the ESP32-CAM's GPIO pins to visually broadcast the system's current state (e.g., Idle/Scanning vs. Target Locked).
- **Power Distribution & Interconnects**
 - **Regulated Power Hub (5V DC Adapter & Breadboard):** A dedicated, hardwired 5V power source routed through a breadboard to ensure stable voltage delivery. This strictly separates the high-current draw of the servos from the logic-level signals of the ESP32 and TTL cable, preventing brownouts and signal degradation.
 - **Data Tether (USB-to-TTL Serial Cable):** Acts as the real-time nervous system of the project, providing a direct, zero-lag communication bridge between the Python AI pipeline on the compute node and the C++ firmware on the ESP32.
 - **Logic Routing (DuPont Jumper Wires):** Standardized female-to-female interconnects for establishing a common ground and routing GPIO logic signals from the microcontroller to the servos.

3. Technical Approach (Hybrid Methodology)

To achieve real-time computational performance (>30 FPS) and accurate targeting, this project explores a hybrid architecture. The system is strictly divided into a Deep Learning Vision module (running locally on a GPU) and a Classical Engineering control module executing the physical motor kinematics. This structure allows for a practical investigation into how modern AI and classical control theory interact.

3.1 Deep Learning Pipeline (Vision & Tracking)

The AI pipeline handles detecting the human subject, extracting skeletal joints, and maintaining a persistent lock on the target across continuous frames.

- **Pose Estimation Benchmarking (Target Acquisition):** To select the optimal vision model, I will benchmark **YOLOv8-Pose** (optimized for GPU inference) against **MediaPipe BlazePose** (optimized as a CPU baseline). This evaluation will be conducted in two phases to guarantee both mathematical accuracy and hardware viability:
 - **Phase 1: Theoretical Accuracy (COCO-Pose):** To objectively measure spatial accuracy without the massive overhead of manually labeling thousands of keypoints, the models will be evaluated against a subset of the industry-standard **COCO-Pose dataset**. This provides the baseline metrics for **Precision, Recall, F1-Score, and mAP**, mathematically justifying the model's core intelligence.
 - **Phase 2: Hardware-in-the-Loop Validation:** To mitigate "Domain Shift" (where models fail in real-world conditions despite high theoretical scores), the models will be tested directly against the live HD stream of the local tracking webcam. Because Ground Truth labels do not exist for a live stream, this phase will strictly evaluate **Inference Latency (FPS)** on the local RTX 4060 GPU, and monitor **Confidence Score Degradation** caused by the camera's sensor noise and environmental lighting.
- **Multi-Object Tracking (Target Persistence & Ablation Study):** Because pose estimation models inherently suffer from frame-to-frame amnesia, the extracted keypoint coordinates must be processed through a Multi-Object Tracking (MOT) algorithm. To satisfy the project's experimentation requirement, I will conduct a comparative ablation study between **ByteTrack** (a high-speed, purely mathematical tracker) and **DeepSORT** (a tracker that integrates a secondary Convolutional Neural Network to analyze visual appearance).
 - **Evaluation Dataset (Ground Truth Generation):** To mathematically test these trackers and fulfill the minimum 500-sample dataset requirement, I will engineer a custom Evaluation Dataset. I will record a continuous video of myself moving with intentional environmental occlusions (e.g., walking behind furniture). By capturing this with the primary tracking webcam, I am actively testing the algorithms against the actual sensor hardware. The video will be decomposed into sequential frames (>500 samples), and I will use semi-automated annotation software (like CVAT or Roboflow) to establish the "Ground Truth" bounding boxes.

- **Tracking Metrics:** The tracking algorithms will be benchmarked against this Ground Truth Answer Key. Performance will be graded based on **MOTA (Multi-Object Tracking Accuracy)** and the frequency of **ID Switches** to quantitatively prove which algorithm exhibits superior target re-identification after an occlusion event.

3.2 Classical Engineering Pipeline (Computer Vision & Kinematics)

To create a true hybrid architecture, the system utilizes classical computer vision for efficient pre-processing and classical control theory for physical actuation. This prevents over-reliance on computationally expensive neural networks and ensures smooth robotic movement.

- **Classical Computer Vision (The HOG+SVM "Wake-Up" Cascade):** Running heavy deep learning inference continuously on an empty room is computationally wasteful. To optimize system resources and ensure the system is environment-agnostic (requiring no static background modeling), the pipeline utilizes a two-tier cascade:
 - **Level 1 (Detection):** The raw video feed is scanned using a classical **Histogram of Oriented Gradients (HOG)** paired with a **Support Vector Machine (SVM)** classifier. This algorithm scans for the specific gradient vectors (outlines) of human pedestrians, ignoring environmental noise or non-human movement.
 - **Level 2 (Activation):** Only when the HOG+SVM classifier confidently detects a human subject does the system "wake up" the YOLOv8-Pose neural network to begin extracting specific skeletal keypoints. If the subject leaves the frame, the system returns to the low-power HOG scanning state.
- **Signal Filtering (Exponential Moving Average):** Once the deep learning model is active, the raw (X, Y) keypoint coordinates it generates will naturally jitter pixel-by-pixel. Before targeting, these coordinates will pass through a classical Exponential Moving Average (EMA) low-pass filter to mathematically smooth the trajectory:

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1}$$

(Where S_t is the smoothed coordinate, Y_t is the raw AI coordinate, and α is the smoothing factor).

- **Spatial Coordinate Mapping:** Because the primary vision sensor (the laptop webcam) is physically detached from the actuation hardware, the system can no longer rely on simple 2D frame centering. Instead, the 2D pixel coordinates detected

by the camera must be mathematically translated into a 3D spatial coordinate system relative to the turret.

- **Depth Estimation (Z-Axis Proximity):** Standard webcams do not provide native depth (Z-axis) information. To overcome this, the system calculates a pseudo-depth utilizing skeletal keypoint ratios rather than volatile bounding box dimensions. Because overall bounding box height fluctuates drastically based on human posture (e.g., crouching versus standing), the algorithm instead extracts the pixel distance between the left and right shoulder keypoints generated by the YOLOv8-Pose network. Since human bi-acromial (shoulder) width remains physiologically constant regardless of the subject's stance, utilizing the inverse relationship between this specific pixel width and physical distance yields a highly stable pseudo-depth (Z) calculation to accurately position the target in 3D space.
- **Offset Calibration:** The algorithm must account for the physical disparity between the camera's lens and the turret's rotational center. A fixed physical translation vector (representing the X, Y, and Z distance between the laptop and the turret's base) is measured and loaded into the kinematics algorithm to synchronize their coordinate spaces.
- **Dynamic Error Correction (Closed-Loop Calibration):** While the system primarily relies on predictive open-loop kinematics during live tracking, it mitigates physical hardware misalignment (e.g., webcam tilt, lens distortion, or non-planar mounting surfaces) via an automated pre-deployment calibration sequence. Prior to live operation, the turret is programmed to target three predefined spatial coordinates on a static background. The primary vision sensor detects the resulting laser projection within its 2D frame. By calculating the delta between the predicted kinematic location and the actual visual location, the system generates a dynamic error-correction matrix. This matrix is applied to the live inverse kinematics, effectively creating a pseudo-closed-loop system that offsets static physical inaccuracies and prevents drift at longer ranges.
- **Inverse Kinematics (Motor Actuation):** With the camera functioning as a stationary observer, a traditional reactive PID loop lacks the visual feedback necessary to continuously adjust the motors, which would result in uncontrolled oscillation. Therefore, the system replaces reactive control with predictive inverse kinematics.

- Once the 3D target coordinate (X_t, Y_t, Z_t) is mapped relative to the turret's origin, the system predicts the absolute servo angles required to point the laser directly at the target using trigonometric functions. The exact pan and tilt angles are calculated as follows:

$$\theta_{pan} = \arctan\left(\frac{X_t}{Z_t}\right)$$

$$\theta_{tilt} = \arctan\left(\frac{Y_t}{\sqrt{X_t^2 + Z_t^2}}\right)$$

- The local Python script computes these exact angles frame-by-frame and transmits the integer values directly over the USB-to-TTL serial tether. The ESP32 acts strictly as a low-latency edge motor controller, receiving the serial commands and executing them by generating the corresponding PWM signals for the servos.
- **Hardware-Level Servo Mapping:** The trigonometric inverse kinematics yield outputs in radians (e.g., $-\frac{\pi}{2}$ to $\frac{\pi}{2}$). Because the SG90 servos require absolute degree integers bounded between 0 and 180, the Python pipeline executes a final hardware-translation step prior to serial transmission. Assuming the forward-facing focal center is mechanically calibrated to the 90-degree servo position for both axes, the translation is executed as:

$$servo_{pan} = 90 + \left(\theta_{pan} \times \frac{180}{\pi}\right)$$

$$servo_{tilt} = 90 + \left(\theta_{tilt} \times \frac{180}{\pi}\right)$$

To protect the physical actuation limits and prevent mechanical gear stripping, both final integer outputs are algorithmically clamped to a strict $[0, 180]$ operational boundary before being transmitted to the edge motor controller.